Advanced Topics

Notifications

A notification is a message you can display to the user outside of your application's normal UI. When we tell the system to issue a notification, it first appears as an icon in the notification area. To see the details of the notification, the user opens the notification drawer. Both the notification area and the notification drawer are system-controlled areas that the user can view at any time

Creating a notification

The UI information and actions for a notification is specified in a NotificationCompat.Builder object. To create the notification itself, we call NotificationCompat.Builder.build(), which returns a Notification object containing your specifications. To issue the notification, you pass the Notification object to the system by calling NotificationManager.notify().

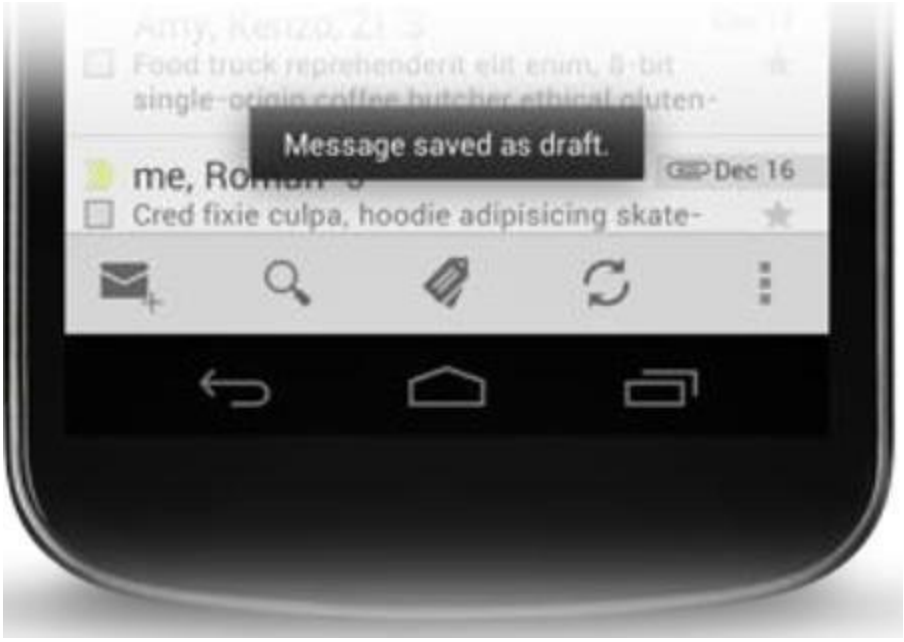
A Notification object must contain the following:

- A small icon, set by setSmallIcon()
- A title, set by setContentTitle()
- Detail text, set by setContentText()

```
NotificationCompat.Builder mBuilder =
    new NotificationCompat.Builder(this)
    .setSmallIcon(R.drawable.notification_icon)
    .setContentTitle("My notification")
    .setContentText("Hello World!");
```

Toast

A toast provides simple feedback about an operation in a small popup. It only fills the amount of space required for the message and the current activity remains visible and interactive. For example, navigating away from an email before you send it triggers a "Draft saved" toast to let you know that you can continue editing later. Toasts automatically disappear after a timeout.



First, instantiate a `Toast` object with one of the `makeText()` methods. This method takes three parameters: the application `Context`, the text message, and the duration for the toast. It returns a properly initialized Toast object. You can display the toast notification with `show()`, as shown in the following example:

```
Context context = getApplicationContext();
CharSequence text = "Hello toast!";
int duration = Toast.LENGTH_SHORT;

Toast toast = Toast.makeText(context, text, duration);
toast.show();
```

This example demonstrates everything you need for most toast notifications. You should rarely need anything else. You may, however, want to position the toast differently or even use your own layout instead of a simple text message. The following sections describe how you can do these things.

You can also chain your methods and avoid holding on to the Toast object, like this:

```
Toast.makeText(context, text, duration).show();
```

Positioning your Toast

A standard toast notification appears near the bottom of the screen, centered horizontally. You can change this position with the `setGravity(int, int, int)` method. This accepts three parameters: a `Gravity` constant, an x-position offset, and a y-position offset.

For example, if you decide that the toast should appear in the top-left corner, you can set the gravity like this:

```
toast.setGravity(Gravity.TOP|Gravity.LEFT, 0, 0);
```

If you want to nudge the position to the right, increase the value of the second parameter. To nudge it down, increase the value of the last parameter.

BroadcastReceiver

A broadcast receiver (short receiver) is an Android component which allows you to register for system or application events. All registered receivers for an event are notified by the Android runtime once this event happens. For example, applications can register for the ACTION_BOOT_COMPLETED system event which is fired once the Android system has completed the boot process. Broadcast Receivers simply respond to broadcast messages from other applications or from the system itself. These messages are sometime called events or intents. For example, applications can also initiate broadcasts to let other applications know that some data has been downloaded to the device and is available for them to use, so this is broadcast receiver who will intercept this communication and will initiate appropriate action. There are following two important steps to make BroadcastReceiver works for the system broadcasted intents

- Creating the Broadcast Receiver

Creating the Broadcast Receiver

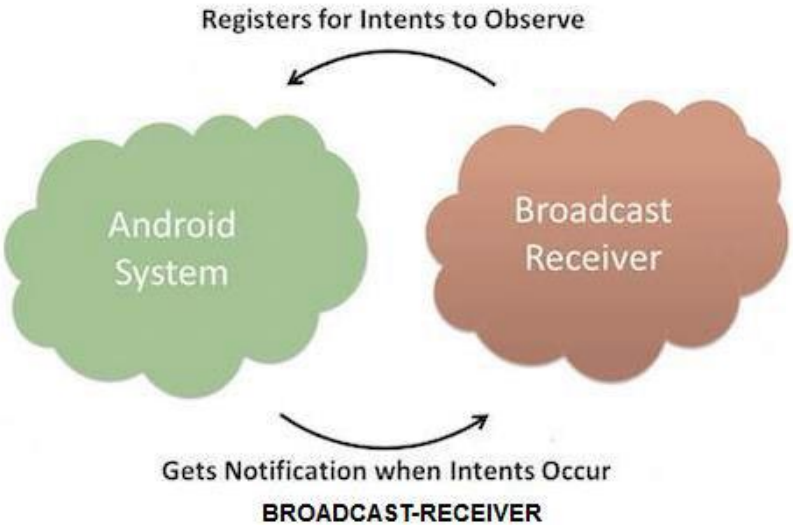
A broadcast receiver is implemented as a subclass of **BroadcastReceiver** class and overriding the onReceive() method where each message is received as a **Intent** object parameter.

```
public class MyReceiver extends BroadcastReceiver {
   @Override
   public void onReceive(Context context, Intent intent) {
      Toast.makeText(context, "Intent Detected.", Toast.LENGTH_LONG).show();
   }
}
```

- Registering Broadcast Receiver

Registering Broadcast Receiver

An application listens for specific broadcast intents by registering a broadcast receiver in *AndroidManifest.xml* file. Consider we are going to register *MyReceiver* for system generated event ACTION_BOOT_COMPLETED which is fired by the system once the Android system has completed the boot process.




```
<receiver android:name=".MyBroadcastReceiver"  android:exported="true">
    <intent-filter>
        <action android:name="android.intent.action.BOOT_COMPLETED"/>
      <action android:name="android.intent.action.INPUT_METHOD_CHANGED" />
    </intent-filter>
</receiver>
```

To declare a broadcast receiver in the manifest, perform the following steps:

1. Specify the <receiver> element in your app's manifest.

```
<receiver
android:name=".MyBroadcastReceiver"  android:exported="true">
    <intent-filter>
        <action
android:name="android.intent.action.BOOT_COMPLETED"/>
        <action
android:name="android.intent.action.INPUT_METHOD_CHANGED" />
    </intent-filter>
```

```
</receiver>
```

The intent filters specify the broadcast actions your receiver subscribes to.

2. Subclass BroadcastReceiver and implement onReceive(Context, Intent). The broadcast receiver in the following example logs and displays the contents of the broadcast:

```
public class MyBroadcastReceiver extends BroadcastReceiver {
        private static final String TAG = "MyBroadcastReceiver";
        @Override
        public void onReceive(Context context, Intent intent) {
            StringBuilder sb = new StringBuilder();
            sb.append("Action: " + intent.getAction() + "\n");
            sb.append("URI: " +
intent.toUri(Intent.URI_INTENT_SCHEME).toString() + "\n");
            String log = sb.toString();
            Log.d(TAG, log);
            Toast.makeText(context, log, Toast.LENGTH_LONG).show();
        }
    }
```

The system package manager registers the receiver when the app is installed. The receiver then becomes a separate entry point into your app which means that the system can start the app and deliver the broadcast if the app is not currently running.

The system creates a new BroadcastReceiver component object to handle each broadcast that it receives. This object is valid only for the duration of the call to onReceive(Context, Intent). Once your code returns from this method, the system considers the component no longer active.

Threads and Handlers

Thread

A Thread is a concurrent unit of execution. It has its own call stack for methods being invoked, their arguments and local variables. Each application has at least one thread running when it is started, the main thread, in the main “ThreadGroup”. The runtime keeps its own threads in the system thread group. There are two ways to execute code in a new thread. We can either subclass Thread and overriding its run() method. Or Construct a new Thread and pass a Runnable to the constructor. In either case, the start() method must be called to actually execute the new Thread.

Some Thread Syntax:

Handler

Handler is part of the Android system's framework for managing threads. A Handler object receives messages and runs code to handle the messages. A Handler allows us to send and process Message and Runnable objects associated with a thread's MessageQueue.

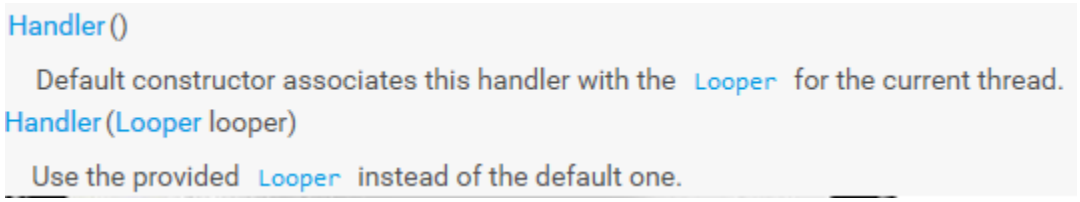
There are two main uses for a Handler:

- to schedule messages and runnables to be executed as some point in the future
- to enqueue an action to be performed on a different thread than your own Scheduling messages is accomplished with the following methods.
    - post(Runnable),
    - postAtTime(Runnable, long),
    - postDelayed(Runnable, long),
    - sendEmptyMessage(int),
    - sendMessage(Message),
    - sendMessageAtTime(Message, long), and
    - sendMessageDelayed(Message, long)

The post versions allow you to enqueue Runnable objects to be called by the message queue when they are received; the sendMessageversions allow you to enqueue a Message object containing a bundle of data that will be processed by the Handler's handleMessage(Message) method (requiring that you implement a subclass of Handler). When posting or sending to a Handler, you can either allow the item to be processed as soon as the message queue is ready to do so, or specify a delay before it gets processed or

absolute time for it to be processed. The latter two allow you to implement timeouts, ticks, and other timing-based behavior.

Some Handler Syntax:



AsyncTask

AsyncTask enables proper and easy use of the UI thread. This class allows to perform background operations and publish results on the UI thread without having to manipulate threads and/or handlers.

AsyncTask is designed to be a helper class around `Thread` and `Handler` and does not constitute a generic threading framework. AsyncTasks should ideally be used for short operations (a few seconds at the most.) If you need to keep threads running for long periods of time, it is highly recommended you use the various APIs provided by the `java.util.concurrent` package such as `Executor`, `ThreadPoolExecutor` and `FutureTask`.

An asynchronous task is defined by a computation that runs on a background thread and whose result is published on the UI thread. An asynchronous task is defined by 3 generic types, called `Params`, `Progress` and `Result`, and 4 steps, called `onPreExecute`, `doInBackground`, `onProgressUpdate` and `onPostExecute`.

AsyncTask's generic types

The three types used by an asynchronous task are the following:

1. `Params`, the type of the parameters sent to the task upon execution.
2. `Progress`, the type of the progress units published during the background computation.
3. `Result`, the type of the result of the background computation.

Not all types are always used by an asynchronous task. To mark a type as unused, simply use the type `Void`:

```
private class MyTask extends AsyncTask<Void, Void, Void> { ... }
```

The 4 steps

When an asynchronous task is executed, the task goes through 4 steps:

1. `onPreExecute()`, invoked on the UI thread before the task is executed. This step is normally used to setup the task, for instance by showing a progress bar in the user interface.
2. `doInBackground(Params...)`, invoked on the background thread immediately after `onPreExecute()` finishes executing. This step is used to perform background computation that can take a long time. The parameters of the asynchronous task are passed to this step. The result of the computation must be returned by this step and will be passed back to the last step. This step can also use `publishProgress(Progress...)` to publish one or more units of progress. These values are published on the UI thread, in the `onProgressUpdate(Progress...)` step.
3. `onProgressUpdate(Progress...)`, invoked on the UI thread after a call to `publishProgress(Progress...)`. The timing of the execution is undefined. This method is used to display any form of progress in the user interface while the background computation is still executing. For instance, it can be used to animate a progress bar or show logs in a text field.
4. `onPostExecute(Result)`, invoked on the UI thread after the background computation finishes. The result of the background computation is passed to this step as a parameter.

Alarm Manager

Alarms (based on the AlarmManager class) gives a way to perform time-based operations outside the lifetime of your application. For example, we could use an alarm to initiate a long-running operation, such as starting a service once a day to download a weather forecast.

Alarms have these characteristics:

- They let us fire Intents at set times and/or intervals.
- We can use them in conjunction with broadcast receivers to start services and perform other operations.
- They operate outside of your application, so we can use them to trigger events or actions even when your app is not running, and even if the device itself is asleep.
- They help us to minimize your app's resource requirements. We can schedule operations without relying on timers or continuously running background services.

Note: For timing operations that are guaranteed to occur during the lifetime of your application, instead consider using the Handler class in conjunction with Timer and Thread. This approach gives Android better control over system resources.

Alarm Types: There are two general clock types for alarms: "elapsed real time" and "real time clock" (RTC). Elapsed real time uses the "time since system boot" as a reference, and real time clock uses UTC (wall clock) time. This means that elapsed real time is suited to setting an alarm

based on the passage of time (for example, an alarm that fires every 30 seconds) since it isn't affected by time zone/locale. The real time clock type is better suited for alarms that are dependent on current locale. Both types have a "wakeup" version, which says to wake up the device's CPU if the screen is off. This ensures that the alarm will fire at the scheduled time. This is useful if your app has a time dependency- for example, if it has a limited window to perform a particular operation. If you don't use the wakeup version of your alarm type, then all the repeating alarms will fire when your device is next awake.

Here is the list of types:

- `ELAPSED_REALTIME` —Fires the pending intent based on the amount of time since the device was booted, but doesn't wake up the device. The elapsed time includes any time during which the device was asleep.
- `ELAPSED_REALTIME_WAKEUP` —Wakes up the device and fires the pending intent after the specified length of time has elapsed since device boot.
- `RTC` —Fires the pending intent at the specified time but does not wake up the device.
- `RTC_WAKEUP` —Wakes up the device to fire the pending intent at the specified time.

Networking in Android

Android lets your application connect to the internet or any other local network and allows you to perform network operations. A device can have various types of network connections. This chapter focuses on using either a Wi- Fi or a mobile network connection.

Checking Network Connection

Before you perform any network operations, you must first check that are you connected to that network or internet e.t.c. For this android providesConnectivityManager class. You need to instantiate an object of this class by calling getSystemService() method. Its syntax is given below

```
ConnectivityManager check =(ConnectivityManager)
this.context.getSystemService(Context.CONNECTIVITY_SERVICE);
```

Once you instantiate the object of ConnectivityManager class, you can usegetAllNetworkInfo method to get the information of all the networks. This method returns an array of NetworkInfo. So you have to receive it like this.

```
NetworkInfo[] info =check.getAllNetworkInfo();
```

The last thing you need to do is to check Connected State of the network. Its syntax is given below –

```
for(inti=0;i<info.length;i++){
        if(info[i].getState()==NetworkInfo.State.CONNECTED){
        Toast.makeText(context,"Internet is connected Toast.LENGTH_SHORT).show();
        }
}
```

Apart from this connected states, there are other states a network can achieve. They are listed below:

Sr.No	State
1	Connecting
2	Disconnected
3	Disconnecting
4	Suspended
5	Unknown

Performing Network Operations After checking that you are connected to the internet, you can perform any network operation. Here we are fetching the html of a website from a url. Android provides HttpURLConnection and URL class to handle these operations. You need to instantiate an object of URL class by providing the link of website. Its syntax is as follows –

```
String link ="http://www.google.com";
```

URL url=new URL(link);

After that you need to call openConnection method of url class and receive it in a HttpURLConnection object. After that you need to call the connect method of HttpURLConnection class. HttpURLConnection conn =(HttpURLConnection)url.openConnection();
conn.connect();

And the last thing you need to do is to fetch the HTML from the website. For this you will use InputStream and BufferedReader class. Its syntax is given below –

```
InputStreamis=conn.getInputStream();
BufferedReader reader =new BufferedReader(newInputStreamReader(is,"UTF-8"));
StringwebPage="",data="";
while((data =reader.readLine())!=null){
        webPage+= data +"\n";
}
```

Apart from this connect method, there are other methods available in HttpURLConnection class. They are listed below –

Sr.No	Method & description
1	**disconnect()** This method releases this connection so that its resources may be either reused or closed
2	**getRequestMethod()** This method returns the request method which will be used to make the request to the remote HTTP server
3	**getResponseCode()** This method returns response code returned by the remote HTTP server
4	**setRequestMethod(String method)** This method Sets the request command which will be sent to the remote HTTP server
5	**usingProxy()** This method returns whether this connection uses a proxy server or not